



School of Computer Science

COMP 8347 “Internet Applications and Distributed
Systems”

Summer 2026

Week 8

Parallel & Asynchronous Programming in Python

DR. OLENA SYROTKINA



University of Windsor

Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- Introduction: Why Parallel and Asynchronous Programming?
- Concurrency Concepts
- Threading (Concurrency for I/O-Bound Work)
- Multiprocessing (Parallelism for CPU-Bound Work)
- Asyncio (Modern Asynchronous I/O)
- Parallel & Asynchronous Programming in Django

Introduction: Why Parallel and Asynchronous Programming?

Modern software applications rarely do just *one thing at a time*. They download data, serve multiple users, process files, and update interfaces – all simultaneously. To keep up with these demands, we use **parallel** and **asynchronous** programming to make our programs faster, more responsive, and efficient.

The Problem

Without concurrency, programs execute one instruction after another. If one task (e.g. downloading a large file) takes a long time, the entire program waits. Users experience lag, and CPU resources often sit idle.

The Solutions in Python

Approach	When to Use	Typical Example
Threading	For concurrent I/O tasks (waiting on files, networks, APIs)	Reading 10 URLs at once
Multiprocessing	For CPU-heavy computations	Image processing, simulations
Asyncio	For asynchronous I/O with coroutines	Thousands of concurrent web requests



Introduction: Why Parallel and Asynchronous Programming?

The Solutions in Python

Approach	When to Use	Typical Example
Threading	For concurrent I/O tasks (waiting on files, networks, APIs)	Reading 10 URLs at once
Multiprocessing	For CPU-heavy computations	Image processing, simulations
Asyncio	For asynchronous I/O with coroutines	Thousands of concurrent web requests

Each of these techniques lets Python handle *multiple tasks at once*, but in different ways:

- ✓ **Threading** uses shared memory and is best for tasks that wait often.
- ✓ **Multiprocessing** uses multiple CPU cores for true parallel execution.
- ✓ **Asyncio** is event-driven, lightweight, and ideal for high-concurrency I/O.

Why It Matters

- ✓ Faster execution on multi-core machines
- ✓ Better responsiveness in GUIs and web apps
- ✓ Efficient use of CPU and network resources
- ✓ Scalability for real-world applications (servers, APIs, data pipelines)

Concurrency Concepts

- **Concurrency** – multiple tasks appear to run at once (tasks interleave).
- **Parallelism** – tasks actually run at the same time on multiple cores.
- **Asynchrony** – using non-blocking I/O so that one task doesn't stop others.
- **GIL (Global Interpreter Lock)** – allows only one thread to execute Python bytecode at a time.

Example 1: Serial Execution

```
import time

def download(i):
    print(f"Downloading file {i}...")
    time.sleep(1)
    print(f"File {i} done!")

start = time.perf_counter()

for i in range(3):
    download(i)

print(f"Serial took {time.perf_counter() - start:.2f}s")
```

```
Downloading file 0...
File 0 done!
Downloading file 1...
File 1 done!
Downloading file 2...
File 2 done!
Serial took 3.26s
```

Threading (Concurrency for I/O-Bound Work)

Threads share memory and run in the same process. Ideal for **waiting tasks** (like downloading files, database queries, or sleeping). Not ideal for heavy CPU tasks due to GIL.

Example 2: Using ThreadPoolExecutor

```
from concurrent.futures import ThreadPoolExecutor
import time

def download(i):
    print(f"Start {i}")
    time.sleep(1)
    print(f"Done {i}")

start = time.perf_counter()

with ThreadPoolExecutor(max_workers=3) as pool:
    pool.map(download, range(3))

print(f"Threads took {time.perf_counter() - start:.2f}s")
```

What happens here:

- ✓ ThreadPoolExecutor(max_workers=3) runs up to 3 threads concurrently.
- ✓ Each download() runs at the same time – all 3 finish in about **1 second**, not 3.

Start 0Start 1Start 2

Done 0Done 1Done 2

Threads took 1.22s

Threading (Concurrency for I/O-Bound Work)

Threads share memory and run in the same process. Ideal for **waiting tasks** (like downloading files, database queries, or sleeping). Not ideal for heavy CPU tasks due to GIL.

Example 2.1: Using ThreadPoolExecutor and flush=True

```
import time

def download(i):
    print(f"Downloading file {i}...", flush=True)
    time.sleep(1)
    print(f"File {i} done!", flush=True)

start = time.perf_counter()

for i in range(3):
    download(i)

print(f"Serial took {time.perf_counter() - start:.2f}s")
```

What happens here:

- ✓ The key addition is `flush=True` in each `print()` – this forces Python to immediately send the output to the console instead of buffering it.
- ✓ Some environments (especially PyCharm or IDLE) delay printing without this, causing multiple prints to appear on the same line.

```
Downloading file 0...
File 0 done!
Downloading file 1...
File 1 done!
Downloading file 2...
File 2 done!
Serial took 3.26s
```


Threading (Concurrency for I/O-Bound Work)

Threads share memory and run in the same process. Ideal for **waiting tasks** (like downloading files, database queries, or sleeping). Not ideal for heavy CPU tasks due to GIL.

Example 3: Download Web Pages

Type: `pip install requests`

```
import requests
from concurrent.futures import ThreadPoolExecutor
import time

urls = ["https://httpbin.org/delay/1" for _ in range(5)]

def fetch(url):
    r = requests.get(url)
    return r.status_code

start = time.perf_counter()

with ThreadPoolExecutor() as ex:
    for code in ex.map(fetch, urls):
        print(code)

print(f"Threads done in {time.perf_counter() - start:.2f}s")
```

What it does:

- ✓ Creates 5 URLs that each delay for 1 second.
- ✓ ThreadPoolExecutor sends 5 requests concurrently (not one after another).
- ✓ You will see all 200 status codes printed quickly.

```
/Users/os/PycharmProjects/PythonProject/
200
200
200
200
200
Threads done in 10.10s
```

Process finished with exit code 0

Multiprocessing (Parallelism for CPU-Bound Work)

Threads don't help CPU-heavy work because of the GIL. Processes do – they have their own Python interpreter and memory.

What is the GIL?

The **Global Interpreter Lock (GIL)** is a **mutex (mutual exclusion lock)** that protects access to Python objects, preventing multiple native threads from executing Python bytecode **at the same time** within a single process.

For example:

Even if you create 10 threads, only **one thread** can execute Python code at a time in the **same interpreter**.

Why does Python have a GIL?

Python's memory management (especially reference counting for garbage collection) is **not thread-safe** by default. The GIL was introduced to **simplify implementation** of CPython (the standard Python interpreter) and **avoid race conditions** without needing complex fine-grained locking.

So, the GIL ensures:

- ✓ Simpler, safer memory handling
- ✓ Less overhead for single-threaded code

But the trade-off is:

- ✓ **Limited true parallelism** for CPU-bound work in threads.

Multiprocessing (Parallelism for CPU-Bound Work)

GIL in Action

Example (CPU-bound)

- ✓ If you run heavy computations (e.g. computing large Fibonacci numbers) in multiple threads, you won't see much speedup – the GIL only allows one thread to run Python bytecode at a time.

Example (I/O-bound)

- ✓ If your threads are waiting for I/O (like web requests or file reads), the GIL is temporarily released while waiting – allowing another thread to run.

That's why threading works well for network or file I/O, but not for CPU-heavy math.

Multiprocessing (Parallelism for CPU-Bound Work)

Example 4: Counting Primes

```
import math
import time

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def count_primes(limit):
    return sum(is_prime(i) for i in range(limit))

start = time.perf_counter()
print(count_primes(100_000))
print(f"Serial took {time.perf_counter() - start:.2f}s")
```

What it does:

- ✓ Counts how many prime numbers are below 100,000.
- ✓ Uses a simple (non-optimized) algorithm

Multiprocessing (Parallelism for CPU-Bound Work)

Example 4.1: Counting Primes - Parallel Version

```
from concurrent.futures import ProcessPoolExecutor
import math
import time

def is_prime(n: int) -> bool:
    if n < 2:
        return False
    # trial division
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def count_primes_part(start_end: tuple[int, int]) -> int:
    start, end = start_end
    return sum(is_prime(i) for i in range(start, end))
```

What it does:

- ✓ You split the range into four chunks, each handled by a separate process.
- ✓ ProcessPoolExecutor runs each chunk on a different CPU core.
- ✓ Each process has its own Python interpreter, so the GIL doesn't apply.

```
def main():
    parts = [
        (2, 50_000),
        (50_000, 100_000),
        (100_000, 150_000),
        (150_000, 200_000),
    ]

    start_t = time.perf_counter()
    with ProcessPoolExecutor() as pool:
        results = pool.map(count_primes_part,
                           parts)  # no lambda here
        total = sum(results)

    print(total)
    print(f"Parallel took {time.perf_counter() - start_t:.2f}s")

if __name__ == "__main__":
    main()
```


Multiprocessing (Parallelism for CPU-Bound Work)

When to Use Which?

Task Type	Best Tool	Reason
Downloading many files	Threads / Asyncio	These tasks spend most of their time waiting for the network or disk (I/O-bound). Threads and asyncio let your program start other downloads while waiting, keeping it responsive and fast.
Parsing thousands of JSONs	Threads	JSON parsing involves many small I/O and CPU operations. The GIL is released during I/O, and threads are lightweight enough to handle many files efficiently.
Heavy math loops	Multiprocessing	These are CPU-bound tasks (e.g., matrix multiplications, simulations). Because of the GIL , only multiprocessing can use multiple CPU cores for true parallel computation.
Mixing I/O + CPU	Async + ProcessPool	For workloads combining I/O (network, file) and CPU (processing or analysis), use Asyncio for the I/O parts and a ProcessPoolExecutor for the CPU-heavy functions. This gives you the best of both worlds.

Asyncio (Modern Asynchronous I/O)

Instead of threads, asyncio uses an event loop. It runs one task at a time, but switches between them whenever one waits.

Example 5: Minimal Async Example

```
import asyncio

async def say_hello():
    print("Hello")
    await asyncio.sleep(1)
    print("World")

asyncio.run(say_hello())
```

```
/Users/os/PycharmProjects/PythonProject/
```

```
Hello
```

```
World
```

```
Process finished with exit code 0
```

Explanation:

- ✓ `async def` defines an **asynchronous coroutine**.
- ✓ `await asyncio.sleep(1)` pauses the coroutine for 1 second **without blocking** the event loop – other async tasks could run during this time.
- ✓ `asyncio.run()` starts the event loop and runs the coroutine to completion.

Asyncio (Modern Asynchronous I/O)

Example 5.1: Run Many Tasks (asynchronous concurrency in Python)

```
import asyncio
import random

async def download(i):
    await asyncio.sleep(random.random())    # simulate variable network delay
    print(f"File {i} done")

async def main():
    # create 5 concurrent async tasks
    tasks = [asyncio.create_task(download(i)) for i in range(5)]
    await asyncio.gather(*tasks)    # run them all at once

asyncio.run(main())
```

```
/Users/os/PycharmProjects/PythonProject/
```

```
File 2 done
```

```
File 0 done
```

```
File 1 done
```

```
File 4 done
```

```
File 3 done
```

Explanation:

- ✓ Each `download(i)` runs concurrently, not one by one.
- ✓ `asyncio.sleep()` doesn't block other tasks; it just tells the event loop "I'm waiting."
- ✓ The tasks finish in random order depending on the random sleep durations.

Parallel & Asynchronous Programming in Django

Topics

- ✓ Request/Response lifecycle
- ✓ WSGI vs ASGI
- ✓ The Global Interpreter Lock (GIL)
- ✓ When concurrency matters in web apps

1. Request / Response Lifecycle

Let's remember **how a web request travels through your application** and returns a response.

Steps:

- 1. Client sends a request** (e.g., browser → /home).
- 2. Web server (e.g., Nginx, Apache)** receives it and forwards it to your **Python app**.
- 3. Framework (Django, Flask, FastAPI, etc.)** processes the request:
 1. Parses HTTP headers.
 2. Calls the correct *view* or *route function*.
 3. Runs business logic or queries the database.
- 4. The framework builds an HTTP response** and sends it back to the client.

Parallel & Asynchronous Programming in Django

2. WSGI vs ASGI

These define how Python web apps talk to servers – they are standards, not frameworks.

Feature	WSGI (Web Server Gateway Interface)	ASGI (Asynchronous Server Gateway Interface)
Introduced	~2003 (Python 2.x era)	~2017 (for async Python 3.x)
Supports	Synchronous apps only	Async + Sync apps
Works well with	Flask, Django (pre-3.0)	FastAPI, Django ≥ 3.0 , Starlette
Handles	One request per thread/process	Many concurrent async tasks
Example server	Gunicorn (sync workers)	Uvicorn, Hypercorn

Parallel & Asynchronous Programming in Django

3. The Global Interpreter Lock (GIL)

We discussed this earlier – it's a mutex in CPython that allows only one thread to execute Python code at a time.

- ✓ Fine for I/O-bound tasks (network, DB).
- ✓ A bottleneck for CPU-bound work (math loops, image processing).
- ✓ Web frameworks usually avoid this by using multiple processes or async I/O.

In web servers, GIL limits per-process threading, but you can still **run many worker processes**.

4. When Concurrency Matters in Web Apps

Concurrency improves performance when many users connect at once.

Scenario	Why Concurrency Helps
Serving thousands of HTTP requests	Each request might wait for DB or API calls — async avoids blocking
Real-time notifications (chat, stock updates)	Requires handling many open connections simultaneously
Background tasks (email sending, caching)	These can run in separate threads/processes
File uploads / downloads	Async streaming keeps server responsive

Parallel & Asynchronous Programming in Django

Step 1: Create a new folder & virtual environment

```
mkdir pyconcurrency_web && cd pyconcurrency_web
python3 -m venv .venv
source .venv/bin/activate    # on Windows: .venv\Scripts\activate
```

Step 2: Install packages

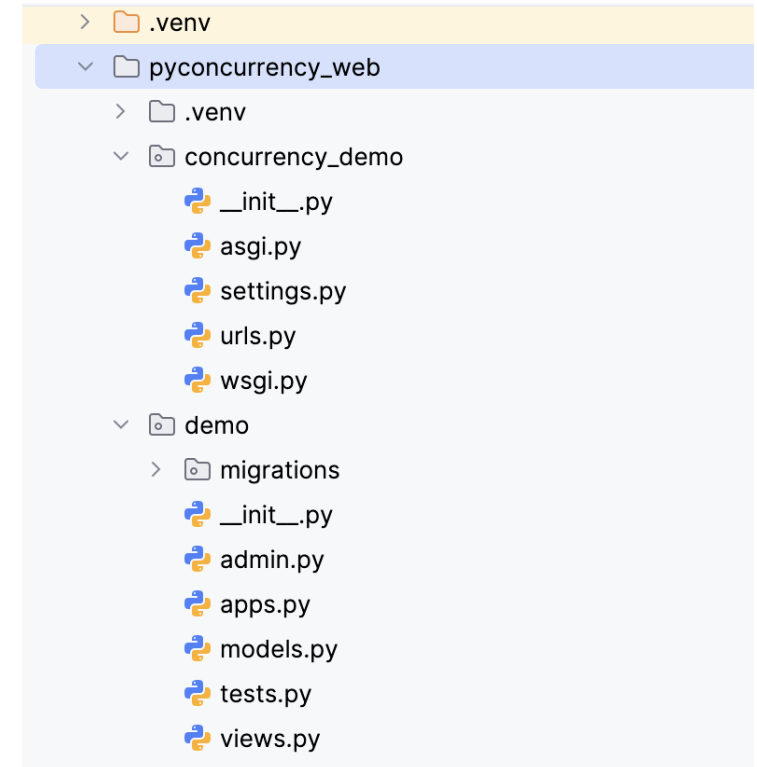
```
pip install "django>=4.2" gunicorn uvicorn
```

Step 3: Create the Django project

```
django-admin startproject concurrency_demo .
python manage.py startapp demo
```

asgi.py – ASGI entrypoint (for Uvicorn/Hypercorn)

wsgi.py – WSGI entrypoint (for Gunicorn sync workers)



Parallel & Asynchronous Programming in Django

Step 4: Enable the demo app

Open `concurrency_demo/settings.py` and add 'demo' to `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'demo',  
]
```

```
from django.contrib import admin  
from django.urls import path  
from demo.views import slow_sync, slow_async  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('slow-sync/', slow_sync), # synchronous view  
    (WSGI-friendly)  
    path('slow-async/', slow_async), # asynchronous view  
    (ASGI-native)  
]
```

Step 5: Add URLs for sync and async endpoints

Edit `concurrency_demo/urls.py`:

Parallel & Asynchronous Programming in Django

Step 6: Implement the views

Edit demo/views.py:

```
# demo/views.py
import time
import asyncio
from django.http import HttpResponse

# --- Synchronous view (blocks the worker for ~3s) ---
def slow_sync(request):
    time.sleep(3)  # simulate slow I/O
    return HttpResponse("sync: Done after 3s")

# --- Asynchronous view (non-blocking sleep for ~3s) ---
async def slow_async(request):
    await asyncio.sleep(3)  # lets the event loop serve
    other requests
    return HttpResponse("async: Done after 3s")
```

- ✓ **slow_sync** will block a thread/process for ~3s.
- ✓ **slow_async** yields control to the event loop (await), allowing other requests to progress in the same worker when you run under an ASGI server.

Parallel & Asynchronous Programming in Django

Step 7: Run with Django's dev server (quick local test)

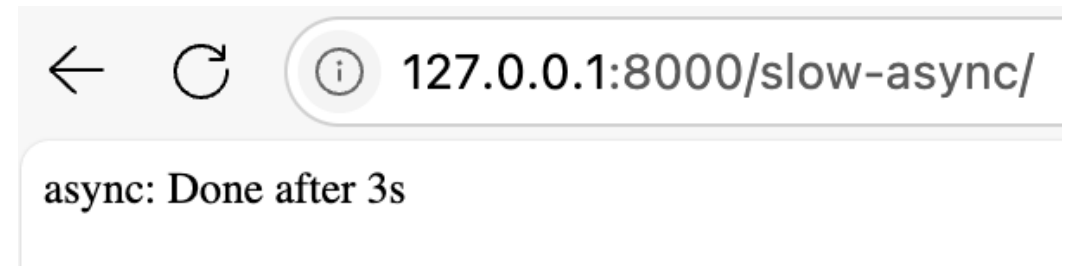
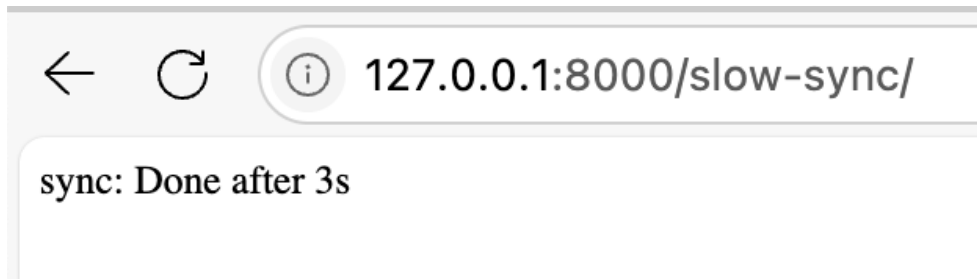
```
python manage.py runserver
```

Open in your browser:

<http://127.0.0.1:8000/slow-sync/>

<http://127.0.0.1:8000/slow-async/>

Both endpoints will “take 3s,” but the async one only shows its advantage under an **ASGI** server when multiple requests come in at once.



Parallel & Asynchronous Programming in Django

When to Use Async Views?

Async views is good whenever your app does **a lot of waiting** – for example, waiting for network responses (APIs), database queries, file reads/writes, web scraping, or long-running I/O tasks.

Because async code *doesn't block the thread* while waiting, your server can handle **many requests at once**, instead of being stuck waiting on one.

Scenario	Why Async Helps
Calling external APIs	You can request data from multiple APIs (e.g., weather, prices, etc.) in parallel without blocking.
Downloading many files or images	Each download “sleeps” while waiting for the network – async lets others run in that time.
Chat or notifications system	Real-time messaging works smoothly because async allows thousands of open connections.
Streaming data or websockets	Async is required for WebSocket support – used in live dashboards or chat.
Microservices communicating over HTTP	Async views allow efficient inter-service communication.

Parallel & Asynchronous Programming in Django

When to Keep It Synchronous?

If your code is mostly **CPU-bound** (heavy math, data processing, etc.), async doesn't help – it still runs on a single CPU core.

Scenario	Better Option
Complex calculations or simulations	Use multiprocessing or a background task (e.g., Celery).
Heavy database operations	Databases often have synchronous drivers (though async ones are emerging).
Simple web pages or admin panels	Sync views are simpler and fine for low concurrency.

In Django ...

Async views can be used in:

- ✓ **Modern Django REST APIs** (with async def endpoints)
- ✓ **WebSocket apps** (via Django Channels)
- ✓ **Streaming responses** (e.g., progress updates)
- ✓ **External API aggregators**
- ✓ **IoT dashboards or monitoring tools**

But remember:

They only give real performance gains if your app runs on an **ASGI server** like **Uvicorn**, **Daphne**, or **Hypercorn** – not on traditional WSGI servers like Gunicorn (sync workers).

Parallel & Asynchronous Programming in Django

Step 8: Run with Gunicorn (WSGI, sync workers)

Gunicorn uses the **WSGI** entrypoint and sync workers by default – great for classic sync Django/Flask apps:

pip install gunicorn

```
python -m gunicorn concurrency_demo.wsgi:application \  
--pythonpath pyconcurrency_web \  
--workers 4 --bind 0.0.0.0:8000
```

Open in your browser:

<http://127.0.0.1:8000/slow-sync/>

<http://127.0.0.1:8000/slow-async/>

Hitting /slow-sync/ uses up a worker for ~3s.

Hitting /slow-async/ still runs, but it is adapted through Django's ASGI→WSGI “sync shim”, so each request uses a worker for ~3s as well.

Parallel & Asynchronous Programming in Django

What we did...

We ran our Django project using Gunicorn, like this:

```
gunicorn concurrency_demo.wsgi:application --workers 4 --bind 0.0.0.0:8000
```

This means:

We started **Gunicorn**, a popular web server for Django/Flask.

It used **4 workers**, which means it can handle 4 requests at the same time.

It runs using **WSGI** – the old, synchronous (one-request-at-a-time) way.

What happens with /slow-sync/

If you open <http://127.0.0.1:8000/slow-sync/>:

That request takes **3 seconds**.

While it's waiting, the worker that handles it is **busy** and **can't take another request**.

With 4 workers, only 4 requests can run at the same time – the 5th one must wait.

What happens with /slow-async/

If you open <http://127.0.0.1:8000/slow-async/>:

Even though it's defined as an async view, Gunicorn (using WSGI) **doesn't support true async**.

Django automatically wraps it in a small layer ("**sync shim**") that makes async look like normal sync code.

So this view also **blocks a worker for 3 seconds**, just like the sync one.

Parallel & Asynchronous Programming in Django

Server	Type	Async Supported?	Effect
Gunicorn (WSGI)	Synchronous	✗ No	Each request blocks a worker until done
Uvicorn (ASGI)	Asynchronous	✓ Yes	Async views can run many requests at once

Step 9: Run with Uvicorn (ASGI, async event loop)

Uvicorn uses the **ASGI** entrypoint and can serve async views concurrently in the **same** worker:

```
pip3 install uvicorn
python3 -m uvicorn concurrency_demo.asgi:application \
  --app-dir pyconcurrency_web \
  --host 0.0.0.0 \
  --port 8000
```

Parallel & Asynchronous Programming in Django

Step 9: Run with Uvicorn (ASGI, async event loop)

Uvicorn uses the **ASGI** entrypoint and can serve async views concurrently in the **same** worker:

```
pip3 install uvicorn
python3 -m uvicorn concurrency_demo.asgi:application \
    --app-dir pyconcurrency_web \
    --host 0.0.0.0 \
    --port 8000
```

With just **1 worker**, make **several** requests to `/slow-async/` quickly (open multiple tabs, or use a small script/cURL loop). You will see they complete in parallel (roughly 3s total for many), because `await asyncio.sleep` is non-blocking. Do the same for `/slow-sync/` - requests will queue because the sync view blocks.

Parallel & Asynchronous Programming in Django

What You Were Testing?

That test was designed to visually prove how Django behaves under concurrent requests for synchronous vs. asynchronous views.

You sent three requests at the same time to:

/slow-async/ — your async view (using `await asyncio.sleep(3)`)

/slow-sync/ — your sync view (using `time.sleep(3)`)

Each request takes ~3 seconds to finish.

So when you ran this under **Uvicorn**, both async and sync finished around 3 s, because Uvicorn uses a **thread pool** for sync views - avoiding the blocking problem.